# CRICBUZZ LIVE ANALYSIS

**SUBMITTED BY:**
SAMUEL JAYSON

**Course:**
DATA ANALYTICS

Date:14/09/2025

**MODULES**

- RAPID API – for web scrapping
- MYSQL - For Data Cleaning, EDA and KPI
- STREALIT – For Building Dashboard

## Introduction

The Cricbuzz Live Match Analysis dashboard is designed to provide real-time insights and historical context for cricket matches. It aggregates data on matches, teams, venues, and series to offer a comprehensive view of cricket trends. Users can explore upcoming, live, and recent matches interactively. The dashboard highlights key performance indicators such as total matches, top teams, frequently used venues, and series statistics. By combining historical match data with interactive visualizations, it allows analysts, fans, and researchers to understand patterns in team performances and venue usage. The project bridges data analytics and sports insights, making cricket statistics accessible and actionable. It also provides a user-friendly interface to explore match outcomes, toss results, and series popularity. Overall, it enhances decision-making and engagement for cricket enthusiasts.

## Methodology:

The project uses structured CSV datasets containing information on matches, teams, series, and venues. Data preprocessing includes checking for missing values, ensuring consistent column formats, and converting dates to proper datetime objects. Streamlit serves as the interactive web framework, while Plotly is used for creating rich and responsive visualizations. KPIs such as total matches, teams, series, venues, and match categories are calculated for quick insights. Visualizations include bar charts, pie charts, line graphs, and scatter plots to show trends over years, venue popularity, top teams, and toss results. Users can filter tables, select specific datasets, and explore detailed team performance metrics. Data caching is implemented to improve dashboard performance and reduce loading time. The methodology

ensures accurate, visually appealing, and easy-to-navigate insights for end users.

**Key Findings:**

The dashboard reveals that certain venues consistently host more matches than others, indicating popularity and infrastructure strength. Top-performing teams emerge when analyzing matches played and toss outcomes. Match trends over the years show periods of high and low activity, reflecting tournament schedules and seasonality. Series analysis identifies recurring and high-profile series with significant participation. Live and upcoming matches are highlighted, helping users stay updated on ongoing events. KPI metrics provide a quick snapshot of overall cricket activity. The dashboard also allows users to explore individual team statistics, city-wise match distribution, and series-based performance. Interactive visualizations make it easy to compare different datasets, identify patterns, and make data-driven insights. Overall, the tool offers actionable intelligence for fans, analysts, and cricket researchers.

**Jupyter(MySQL)**

```
# Convert Start Date to datetime
matches['Start Date'] = pd.to_datetime(matches['Start Date'],
errors='coerce')

# Fill missing columns if needed
expected_cols = ['Venue', 'City', 'Team 1', 'Team 2', 'Series',
'Match Category', 'Start Date', 'Toss Winner']
for col in expected_cols:
```

```
    if col not in matches.columns:
        matches[col] = None

# ----------------
# QUERY 1: Total Matches
# ----------------
total_matches = len(matches)
print(f"Total Matches: {total_matches}")

# ----------------
# QUERY 2: Total Teams
# ----------------
total_teams = len(teams)
print(f"Total Teams: {total_teams}")

# ----------------
# QUERY 3: Total Series
# ----------------
total_series = len(series)
print(f"Total Series: {total_series}")

# ----------------
# QUERY 4: Total Venues
# ----------------
total_venues = len(venues)
print(f"Total Venues: {total_venues}")

# ----------------
# QUERY 5: Most Played Venue
# ----------------
most_played_venue = matches['Venue'].mode()[0] if matches['Venue'].notnull().any() else "N/A"
print(f"Most Played Venue: {most_played_venue}")
```

```
# ----------------
# QUERY 6: Upcoming Matches Count
# ----------------
upcoming_matches = matches[matches['Match Category'].str.lower() == 'upcoming'].shape[0]
print(f"Upcoming Matches: {upcoming_matches}")

# ----------------
# QUERY 7: Live Matches Count
# ----------------
live_matches = matches[matches['Match Category'].str.lower() == 'live'].shape[0]
print(f"Live Matches: {live_matches}")

# ----------------
# QUERY 8: Recent Matches Count
# ----------------
recent_matches = matches[matches['Match Category'].str.lower() == 'recent'].shape[0]
print(f"Recent Matches: {recent_matches}")

# ----------------
# QUERY 9: Matches by City
# ----------------
city_counts = matches['City'].value_counts().reset_index()
city_counts.columns = ['City', 'Matches']
fig_city = px.bar(city_counts, x='City', y='Matches', color='Matches', title="Matches by City", color_continuous_scale='Viridis')
fig_city.show()

# ----------------
```

```
# QUERY 10: Top 10 Teams by Matches Played
# - - - - - - - - - - - - - - - - -
team_counts = pd.concat([matches['Team 1'], matches['Team 2']]).value_counts().reset_index()
team_counts.columns = ['Team', 'Matches Played']
fig_team = px.bar(team_counts.head(10), x='Team', y='Matches Played', color='Matches Played', title="Top 10 Teams by Matches Played", color_continuous_scale='Plasma')
fig_team.show()

# - - - - - - - - - - - - - - - - -
# QUERY 11: Matches Over Years
# - - - - - - - - - - - - - - - - -
matches_over_years = matches.groupby(matches['Start Date'].dt.year).size().reset_index(name='Matches')
fig_years = px.line(matches_over_years, x='Start Date', y='Matches', markers=True, title="Matches Over Years")
fig_years.show()

# - - - - - - - - - - - - - - - - -
# QUERY 12: Top 10 Venues by Matches
# - - - - - - - - - - - - - - - - -
venue_counts = matches['Venue'].value_counts().reset_index()
venue_counts.columns = ['Venue', 'Matches']
fig_venues = px.pie(venue_counts.head(10), names='Venue', values='Matches', hole=0.4, title="Top 10 Venues")
fig_venues.show()
```

**Python (Streamlit)**
App.py

```
import streamlit as st
import pandas as pd
import plotly.express as px

# - - - - - - - - - - - - - - - - -
# Load Data
# - - - - - - - - - - - - - - - - -
 @st.cache_data
 def load_data():
    # Load CSVs
    matches = pd.read_csv("cricbuzz_matches.csv")
    teams = pd.read_csv("teams.csv")
    series = pd.read_csv("series.csv")
    venues = pd.read_csv("venues.csv")

    # Ensure expected columns exist
    expected_cols = ['Venue', 'City', 'Team 1', 'Team 2', 'Series',
'Match Category', 'Start Date', 'Toss Winner']
    for col in expected_cols:
        if col not in matches.columns:
           matches[col] = None  # Add missing columns with None

    # Convert Start Date safely
    matches['Start Date'] = pd.to_datetime(matches['Start Date'],
errors='coerce')

    return matches, teams, series, venues

matches, teams, series, venues = load_data()
```

```
# Dictionary for easy table selection
data_tables = {
    "Matches": matches,
    "Teams": teams,
    "Series": series,
    "Venues": venues
}

# ----------------
# Sidebar Navigation
# ----------------
st.sidebar.title("Navigation")
page = st.sidebar.radio("Go to", ["Home", "KPIs & Metrics",
"Visualizations", "User Profile"])

# ----------------
# Helper for Title Styling
# ----------------
def styled_title(text):
    st.markdown(
        f"""
         <div style='background-color: #001F54; padding: 15px;
border-radius: 10px;'>
           <h1 style='text-align: center; color: white;'>{text}</h1>
        </div>
        """,
        unsafe_allow_html=True
    )

# ----------------
# HOME PAGE
# ----------------
if page == "Home":
```

```
styled_title("Cricbuzz Matches Dashboard")
st.markdown("""
Welcome to the **Cricbuzz Cricket Dashboard**!
  This project provides insights into cricket match data including **teams, matches, venues, and series**.
Use this dashboard to track performance, trends, and cricket analytics in an interactive way.

### Features:
- **KPIs & Metrics:**
    View key numbers like total matches, teams, venues, upcoming matches, and more.
    Explore data tables directly and gain a quick overview of cricket trends.

- **Interactive Visualizations:**
    Analyze cricket performance with rich, colorful visualizations.
    Identify top teams, venues, and trends over time using graphs and charts.

- **User Profile:**
    Learn about the dashboard creator, skills, and portfolio projects.
    Gain context on expertise and approach to cricket data analytics.
""")
st.image("https://upload.wikimedia.org/wikipedia/en/4/4e/Cricket_equipment.jpg", use_container_width=True)

# ----------------
# KPIs & Metrics
# ----------------
```

```
elif page == "KPIs & Metrics":
    styled_title("KPIs & Metrics")

    # KPI Metrics
    total_matches = len(matches)
    total_teams = len(teams)
    total_series = len(series)
    total_venues = len(venues)

        most_played_venue = matches['Venue'].mode()[0] if matches['Venue'].notnull().any() else "N/A"
            upcoming_matches = matches[matches['Match Category'].str.lower() == 'upcoming'].shape[0] if matches['Match Category'].notnull().any() else 0
                live_matches = matches[matches['Match Category'].str.lower() == 'live'].shape[0] if matches['Match Category'].notnull().any() else 0
                recent_matches = matches[matches['Match Category'].str.lower() == 'recent'].shape[0] if matches['Match Category'].notnull().any() else 0

    col1, col2, col3, col4 = st.columns(4)
    col1.metric("Total Matches", total_matches)
    col2.metric("Total Teams", total_teams)
    col3.metric("Total Series", total_series)
    col4.metric("Total Venues", total_venues)

    col5, col6, col7, col8 = st.columns(4)
    col5.metric("Most Played Venue", most_played_venue)
    col6.metric("Upcoming Matches", upcoming_matches)
    col7.metric("Live Matches", live_matches)
    col8.metric("Recent Matches", recent_matches)
```

```
    # Quick Stats Table
    st.markdown("### Quick Match Stats")
    st.dataframe(matches.head(10))

    # Select Table to Display
    st.markdown("### Explore Other Tables")
        table_choice = st.selectbox("Choose a Table",
list(data_tables.keys()))
    if st.button("Generate Table"):
        st.write(f"### {table_choice} Table")
        st.dataframe(data_tables[table_choice].head(20))

    # KPI Visualization
        st.markdown("### KPI Visualization - Matches by
Category")
    if matches['Match Category'].notnull().any():
                    category_counts = matches['Match
Category'].value_counts().reset_index()
        category_counts.columns = ['Match Category', 'Count']
        fig = px.bar(
            category_counts,
            x='Match Category',
            y='Count',
            color='Count',
            color_continuous_scale='blues',
            title="Matches by Category"
        )
                    fig.update_layout(plot_bgcolor='white',
paper_bgcolor='white', font_color='black')
        st.plotly_chart(fig, use_container_width=True)

# - - - - - - - - - - - - - - - - -
# VISUALIZATIONS
```

```
# - - - - - - - - - - - - - - - - -
elif page == "Visualizations":
    styled_title("Visualizations")
    st.markdown("### Cricket Insights Visualizations")

    # Matches by City
    city_counts = matches['City'].value_counts().reset_index()
    city_counts.columns = ['City', 'Matches']
    fig1 = px.bar(city_counts, x='City', y='Matches', color='Matches', title="Matches by City", color_continuous_scale='Viridis')
    fig1.update_layout(plot_bgcolor='white', paper_bgcolor='white', font_color='black')
    st.plotly_chart(fig1, use_container_width=True)

    # Top 10 Teams by Matches Played
    team_counts = pd.concat([matches['Team 1'], matches['Team 2']]).value_counts().reset_index()
    team_counts.columns = ['Team', 'Matches Played']
    fig2 = px.bar(team_counts.head(10), x='Team', y='Matches Played', color='Matches Played', title="Top 10 Teams by Matches Played", color_continuous_scale='Plasma')
    fig2.update_layout(plot_bgcolor='white', paper_bgcolor='white', font_color='black')
    st.plotly_chart(fig2, use_container_width=True)

    # Matches Over Years
    over_years = matches.groupby(matches['Start Date'].dt.year).size().reset_index(name='Matches')
    fig3 = px.line(over_years, x='Start Date', y='Matches', markers=True, title="Matches Over Years")
    fig3.update_layout(plot_bgcolor='white', paper_bgcolor='white', font_color='black')
```

```
st.plotly_chart(fig3, use_container_width=True)

# Top 10 Venues
venue_counts = matches['Venue'].value_counts().reset_index()
venue_counts.columns = ['Venue', 'Matches']
fig4 = px.pie(venue_counts.head(10), names='Venue', values='Matches', hole=0.4, title="Top 10 Venues")
fig4.update_layout(plot_bgcolor='white', paper_bgcolor='white', font_color='black')
st.plotly_chart(fig4, use_container_width=True)

# Top 10 Series
series_counts = matches['Series'].value_counts().reset_index()
series_counts.columns = ['Series', 'Matches']
fig5 = px.pie(series_counts.head(10), names='Series', values='Matches', hole=0.4, title="Top 10 Series")
fig5.update_layout(plot_bgcolor='white', paper_bgcolor='white', font_color='black')
st.plotly_chart(fig5, use_container_width=True)

# Matches by Toss Winner
toss_counts = matches['Toss Winner'].value_counts().reset_index()
toss_counts.columns = ['Toss Winner', 'Count']
fig6 = px.bar(toss_counts.head(10), x='Toss Winner', y='Count', color='Count', title="Matches by Toss Winner", color_continuous_scale='Sunset')
st.plotly_chart(fig6, use_container_width=True)

# Matches by Venue
```

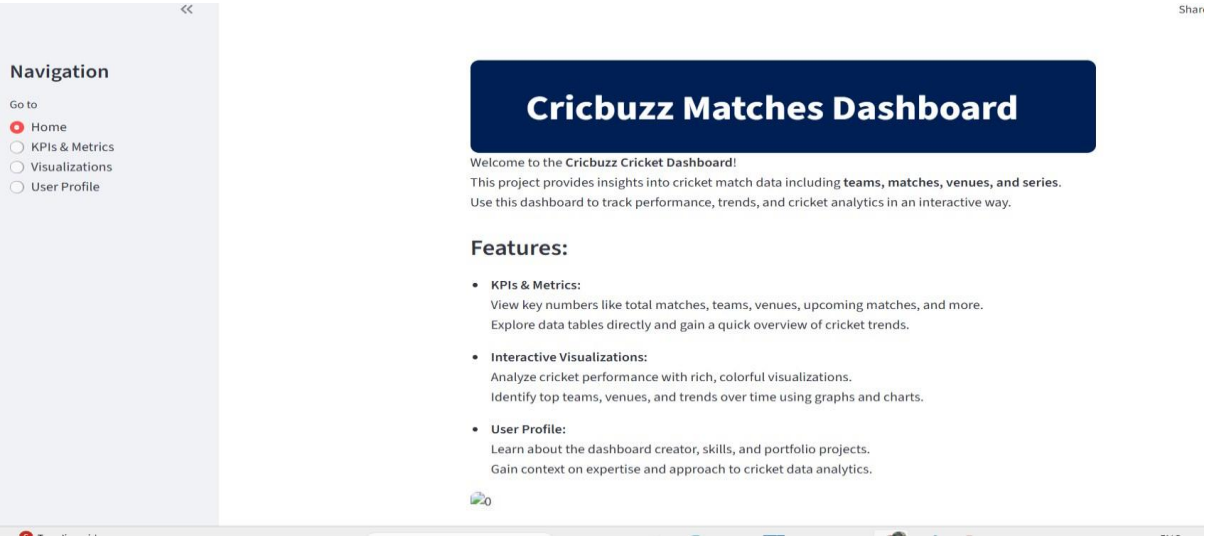
```
        fig7 = px.bar(venue_counts.head(10), x='Venue', y='Matches', color='Matches', title="Matches by Venue", color_continuous_scale='Magma')
    st.plotly_chart(fig7, use_container_width=True)

    # Matches per Team (Detailed)
        team_detailed_counts = pd.concat([matches['Team 1'], matches['Team 2']]).value_counts().reset_index()
    team_detailed_counts.columns = ['Team', 'Total Matches']
     fig8 = px.scatter(team_detailed_counts, x='Team', y='Total Matches', size='Total Matches', color='Total Matches', title="Matches per Team (Detailed)")
                                fig8.update_layout(plot_bgcolor='white', paper_bgcolor='white', font_color='black')
    st.plotly_chart(fig8, use_container_width=True)

# - - - - - - - - - - - - - - - - -
# USER PROFILE
# - - - - - - - - - - - - - - - - -
elif page == "User Profile":
    styled_title("User Profile")
    st.markdown("""
    **Name:** Nithish Rexson
    **Role:** Data Analyst
    **Skills:** Python, Pandas, MySQL, Power BI, Excel
    **Projects:**
    - IPL Win Prediction
    - Flight Delay Analysis
    - Bird Species Distribution
    - Food Waste Management System

    **About Me:**
```

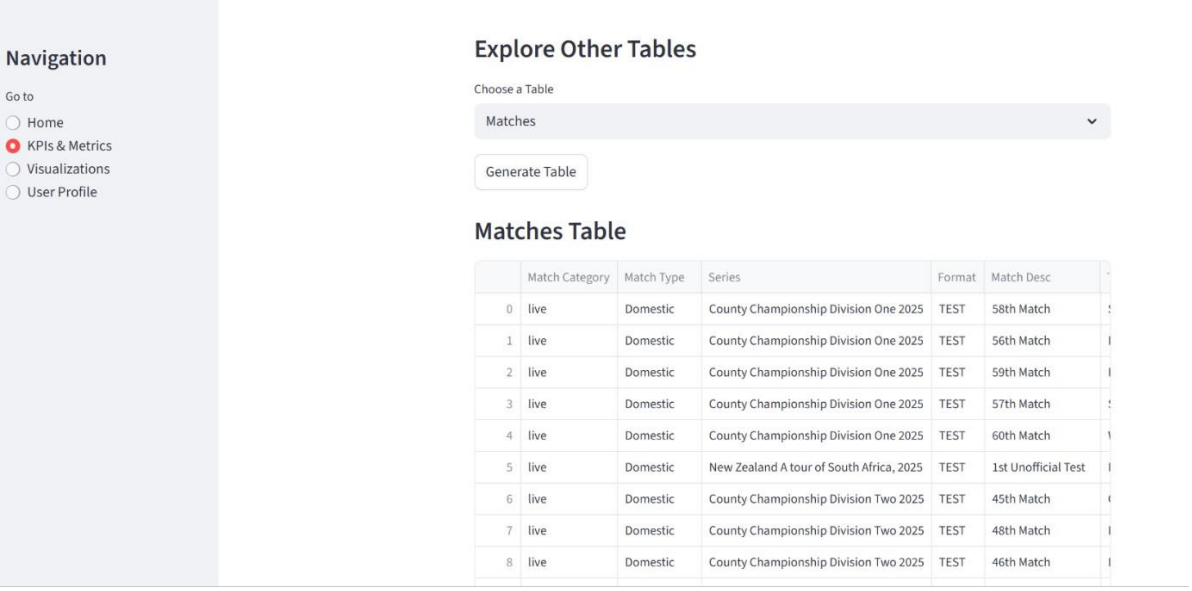
Passionate about data analytics, visualization, and building solutions that create impact.
""")

**Explanation**

This Streamlit app loads cricket match data from CSV files (matches, teams, series, venues) using Pandas, caches it for performance, and provides a sidebar to navigate between four pages: Home, KPIs & Metrics, Visualizations, and User Profile. The Home page shows a dashboard introduction and an image, KPIs & Metrics displays key statistics like total matches, teams, series, venues, and counts of live/upcoming/recent matches along with a bar chart of match categories. The Visualizations page uses Plotly to create interactive charts for matches by city, top teams, matches over years, top venues and series, toss winners, and detailed team match counts. The User Profile page shows personal details and projects. Titles are styled with a dark blue background and white text, and tables can be explored interactively via select boxes. Overall, it provides an interactive, visually appealing dashboard for cricket match analysis.

# Final output:




| | Match Category | Match Type | Series | Format | Match Desc |
|---|---|---|---|---|---|
| 0 | live | Domestic | County Championship Division One 2025 | TEST | 58th Match |
| 1 | live | Domestic | County Championship Division One 2025 | TEST | 56th Match |
| 2 | live | Domestic | County Championship Division One 2025 | TEST | 59th Match |
| 3 | live | Domestic | County Championship Division One 2025 | TEST | 57th Match |
| 4 | live | Domestic | County Championship Division One 2025 | TEST | 60th Match |
| 5 | live | Domestic | New Zealand A tour of South Africa, 2025 | TEST | 1st Unofficial Test |
| 6 | live | Domestic | County Championship Division Two 2025 | TEST | 45th Match |
| 7 | live | Domestic | County Championship Division Two 2025 | TEST | 48th Match |
| 8 | live | Domestic | County Championship Division Two 2025 | TEST | 46th Match |

## Navigation

Go to

- Home
- KPIs & Metrics
- Visualizations
- User Profile

# KPIs & Metrics

| Total Matches | Total Teams | Total Series | Total Venues |
|---|---|---|---|
| 145 | 93 | 29 | 45 |

| Most Played Venue | Upcoming Matches | Live Matches | Recent Matches |
|---|---|---|---|
| Albert P… | 65 | 12 | 68 |

## Quick Match Stats

| | Match Category | Match Type | Series | Format | Match Desc | Tea |
|---|---|---|---|---|---|---|
| 0 | live | Domestic | County Championship Division One 2025 | TEST | 58th Match | Sur |
| 1 | live | Domestic | County Championship Division One 2025 | TEST | 56th Match | Dur |
| 2 | live | Domestic | County Championship Division One 2025 | TEST | 59th Match | Har |
| 3 | live | Domestic | County Championship Division One 2025 | TEST | 57th Match | Son |
| 4 | live | Domestic | County Championship Division One 2025 | TEST | 60th Match | Wor |
| 5 | live | Domestic | New Zealand A tour of South Africa. 2025 | TEST | 1st Unofficial Test | Nev |

## Navigation

Go to

- Home
- KPIs & Metrics
- Visualizations
- User Profile

## KPI Visualization - Matches by Category

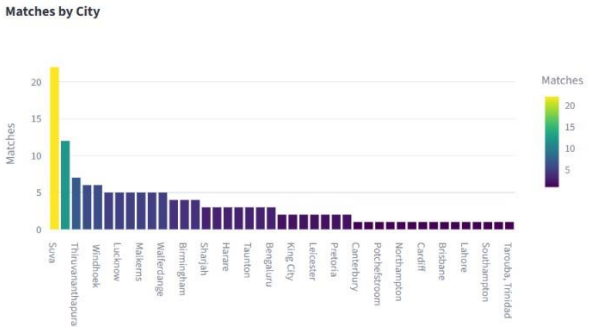
## Navigation

Go to

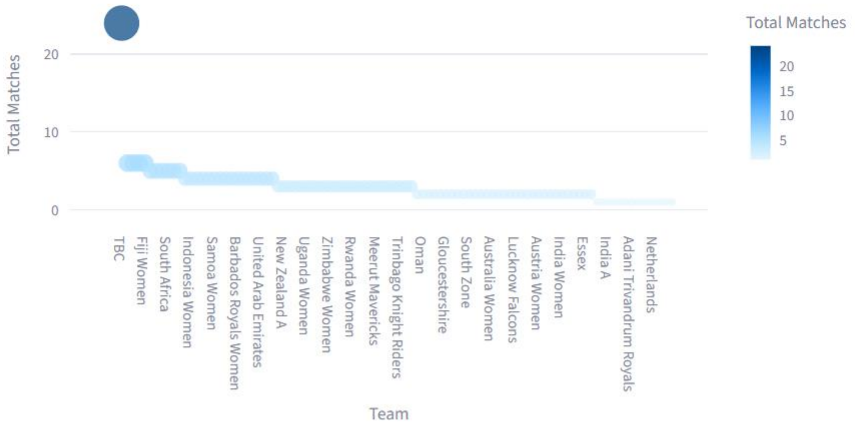
- Home
- KPIs & Metrics
- Visualizations
- User Profile

# Visualizations

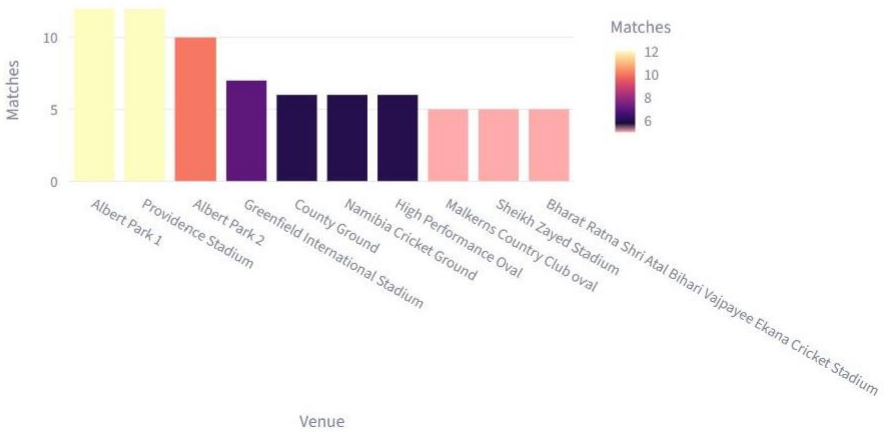
## Cricket Insights Visualizations



**Matches per Team (Detailed)**

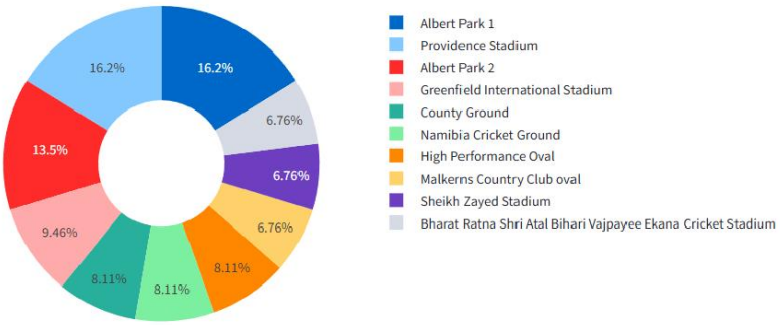


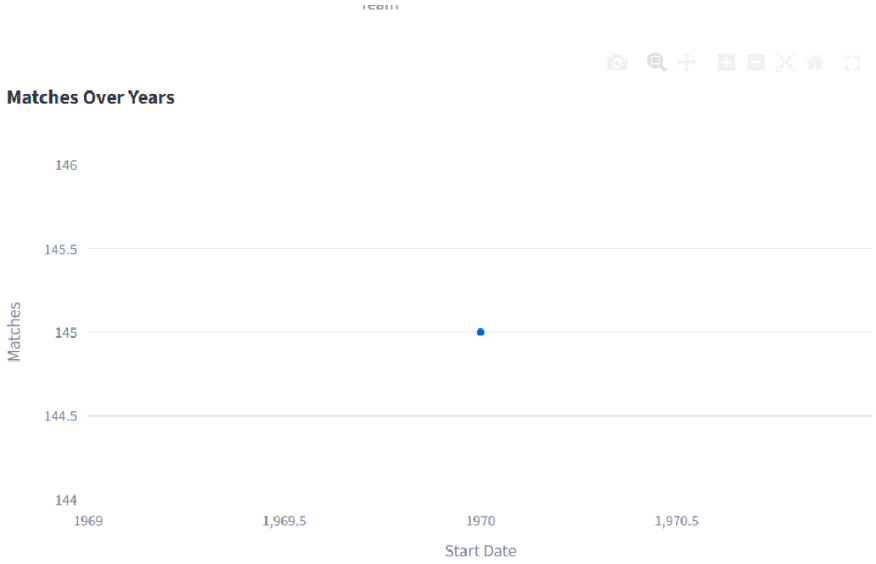

**Matches by Venue**

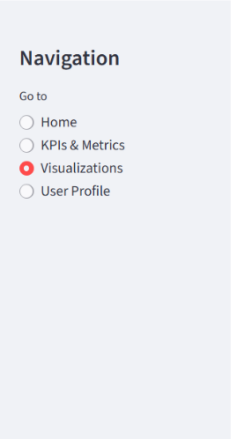


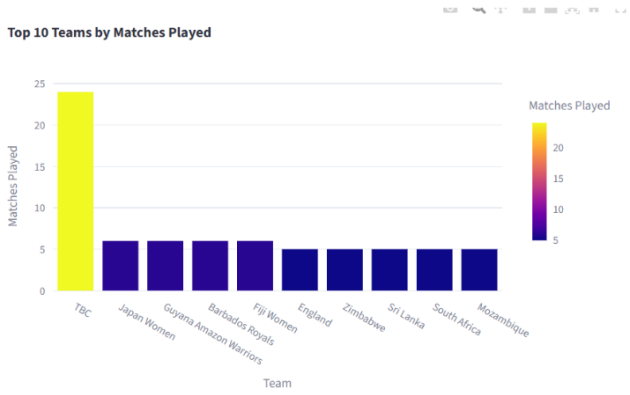
**Top 10 Series**

**Top 10 Venues**






Home Page: The Home page provides a brief introduction to the dashboard, highlighting its purpose—to analyze cricket matches, teams, series, and venues interactively. It includes a welcoming message and an illustrative cricket image to give context to the user.

KPIs & Metrics (Key Insights): This section shows important statistics such as total matches, teams, series, venues, and counts of upcoming, live, and recent matches. It also identifies the most frequently played venue. A bar chart summarizes matches by category, giving users a quick snapshot of cricket trends and key insights from the dataset.

Visualizations: This section offers interactive charts using Plotly for deeper analysis. It visualizes matches by city, top

teams, matches over the years, top venues, series, toss winners, and team-wise match counts. These visualizations help users quickly identify patterns, trends, and distributions in cricket match data, making the analysis more intuitive and visually appealing.

**Conclusion:**

The Cricbuzz Matches Dashboard provides a comprehensive overview of cricket match data, enabling users to explore key insights and trends effectively. By combining KPIs with interactive visualizations, it highlights the performance of teams, venues, and series over time. Users can easily identify patterns such as the most active venues, popular teams, and match distribution across cities and years. The dashboard also tracks upcoming, live, and recent matches, offering a real-time perspective on cricket events. With detailed charts on toss winners, series popularity, and team match counts, it simplifies complex data into actionable insights. Overall, this dashboard demonstrates the power of data analytics and visualization in understanding cricket trends, supporting fans, analysts, and stakeholders in making informed decisions. It is a practical tool for monitoring cricket activities while showcasing interactive and user-friendly data presentation.